Data Pipeline Project — Execution Plan

Scraping + scheduled jobs system, from setup to deployment

A scraper that runs on a schedule, cleans and stores data, and handles real-world messiness (rate limits, malformed data, duplicates). Chosen idea: Job-Posting Aggregator for CS internships.

Project Overview

What It Does

- Scrapes CS internship/job postings from a small set of target sites or job boards
- Cleans and deduplicates listings before storing them
- Runs automatically on a schedule (not manually triggered each time)
- Provides a simple way to query or view the collected postings

Key Design Decision to Make Early

Decide how you'll detect duplicate postings across scrape runs (e.g. hashing title+company+location) and how you'll handle a source site changing its HTML structure without crashing the whole pipeline. These two decisions are your strongest interview talking points for this project.

Tech Stack

Layer	Choice
Scraping	Python (requests + BeautifulSoup, or Scrapy for larger scope)
Scheduling	Celery + Redis, or a simple cron job for a lighter setup
Database	PostgreSQL (or SQLite if keeping it fully local)
Data cleaning	Pandas for normalization/deduplication logic
Frontend (optional)	Simple React or plain HTML page to view/query results
Testing	pytest for scraper logic and data cleaning functions
Deployment	Render or Railway for the scheduled job + a small API to serve data

Week-by-Week Execution

Week 1: Core Scraper

- Pick 2-3 target sources (e.g. a job board, a company careers page, a CS-internship-specific list)
- Build the scraper to extract title, company, location, link, and posting date
- Store raw scraped data in the database with a timestamp for each run
- Test manually: run it once, confirm clean data lands in the database

Week 2: Scheduling and Automation

- Add scheduling so the scraper runs automatically (e.g. every 6 hours)

- Implement deduplication logic so re-runs don't create duplicate entries
- Add basic logging so you can see what happened on each run (new postings found, errors, skipped items)

Week 3: Robustness and Edge Cases

- Handle rate limits and timeouts gracefully (retry with backoff, don't crash the whole job)
- Handle malformed or missing fields (e.g. a listing with no location) without breaking the pipeline
- Write pytest tests for the cleaning/deduplication functions and at least one scraper parsing function
- Document what happens when a source site changes structure (your fallback/alerting approach)

Week 4: Interface, Deployment, Documentation

- Build a simple query interface (a small API endpoint or a lightweight frontend) to browse the collected postings
- Deploy the scheduled job and the query interface (Render/Railway)
- Write a clear README: what it scrapes, how scheduling works, and how deduplication/error-handling works

What to Highlight on Your Resume

- State how often the pipeline runs and roughly how many listings it has processed
- Mention specific edge cases handled (rate limiting, malformed data, deduplication logic)
- Frame it as an automated data pipeline, not just a scraper — this is the difference in how it reads
- Link the GitHub repo and, if deployed, the live query interface

Interview Talking Points This Project Unlocks

- "How do you detect and prevent duplicate entries across scrape runs?"
- "What happens when a source website changes its page structure?"
- "How would you scale this if you needed to scrape 100 sources instead of 3?"
- "Why did you choose a scheduled job over an on-demand scrape?"